FeatureLeakageLens

Interview Defense Document v3.0

Pre-Training Feature Leakage Auditing for Tabular ML

Metric	Value
Checks	7 (6 in v0.1-v0.4, 1 added as 7th)
Checks that can FAIL	2 (temporal_availability, training_future_date_scan)
Checks that WARN	5
Current version	v0.4.0
PyPI published	pip install featureleakagelens
Test suite	23 tests
Output formats	JSON | Markdown | HTML
Core dependencies	NumPy + pandas only
CLI available	Yes -- PASS=0, WARN=1, FAIL=2, INSUFFICIENT_INPUT=3
Truth boundary	Mandatory on all outputs -- flags patterns, not proven leakage

Sidharth Kriplani

Applied LLM Systems Engineer Portfolio

Sourced exclusively from repo files | All claims verified

Label Key

Label	Meaning
[IMPLEMENTED]	Code exists, runs, produces verified artifact
[SIMULATED]	Deliberately synthetic scenario; simulation logic is real code
[PARTIALLY VERIFIED]	Code exists but verification relies on internal consistency
[FUTURE]	Not built -- explicitly absent from the repo

Table of Contents

Section 1 -- The Problem: Why Feature Leakage Is the Worst ML Bug

1.1 The Core Failure Mode

Feature leakage is the single most dangerous silent failure in supervised ML. A model trained on a post-outcome column doesn't look miscalibrated -- it looks exceptional. AUC near 1.0, precision through the roof, validation loss flatlining at epoch 2. Nothing in the training curves signals the problem. The failure surfaces in production when the feature isn't available because it was generated after the event you were predicting. By then, the model has shipped, the business has made decisions on it, and months of development are sunk.

> The paradox of feature leakage: the more severe the leakage, the better the model looks during development. A model with AUC = 0.99 on a credit default task should raise immediate suspicion -- legitimate credit models rarely exceed AUC = 0.78.

1.2 Concrete Examples of Leakage That Ships to Production

Post-Outcome Column: A loan default prediction model is trained with 'days_since_last_payment' as a feature. This column is populated after the loan outcome is known -- for defaulted loans, it shows the time since they stopped paying. It doesn't exist at prediction time (the loan hasn't defaulted yet). The model learns this perfectly: 0 days since last payment = paid = no default. AUC = 0.97. In production, the feature is always NULL. The model collapses to predicting the prior.

Temporal Leakage via Rolling Average: A fraud detection model uses a 7-day rolling transaction average as a feature. The pipeline computes this on the full dataset before the train/test split, so the training rows' averages include information from future (test-period) transactions. The model learns future-looking patterns. Validation AUC = 0.95. Deployed with a proper point-in-time rolling average: AUC = 0.72.

Categorical Proxy: A churn prediction model includes 'account_closure_reason' as a feature. Values include 'moved', 'deceased', 'voluntary_close', 'non_payment_close'. The 'non_payment_close' value has a 100% churn rate. This is not a predictive feature -- it's a direct label proxy. A Pearson correlation scan misses this entirely because it's categorical. The model gets AUC = 0.99 and ships.

ID Leakage: Customer IDs in a credit model happen to be assigned sequentially, and the data was collected during a period when the lending criteria changed (stricter from ID 500000 onward). ID < 500000 correlates strongly with default because those were approved under loose criteria. The model learns the ID range as a proxy for the approval cohort. This is not a business rule -- it's a data collection artifact.

Train/Test Split Leakage: An e-commerce churn model is split by a random 80/20 split across all customers, not by time. Customers who churned in month 6 appear in both train and test (their non-churn months are in train; their churn month might be in test). Features like 'avg_spend_last_3_months' computed before the split include post-churn data for some customers. The split is not properly time-based.

1.3 Why This Is Hard to Catch Without a Tool

The standard ML development workflow has no systematic leakage review step. Data scientists check the features they think to check, in the order they think of them, before moving on to model training. There's no standard checklist, no structured output, and no CI gate. The review is informal and incomplete by design -- it's treated as a pre-flight check, not a quality gate. FeatureLeakageLens makes this review step systematic: 7 checks, structured output attachable to a model card or PR, and a CI exit code that blocks model training on confirmed temporal violations.

Section 2 -- Architecture Deep Dive: All 7 Checks

2.1 Three-Tier Architecture

```
Tier 1 -- Name & Structure (zero-compute, runs on column names only) ■■■ Check 1: Post-outcome name heuristic → WARN ■■■ Check 2: ID / proxy scan → WARN Tier 2 -- Statistical Checks (requires data, O(n*k) per feature) ■■■ Check 3: Target correlation scan → WARN ■■■ Check 4: Categorical proxy scan → WARN ■■■ Check 5: Split distribution scan → WARN or INSUFFICIENT_INPUT Tier 3 -- Temporal Integrity (structural violations, not probabilistic) ■■■ Check 6: Temporal availability → FAIL (not WARN) ■■■ Check 7: Training future date scan → FAIL (not WARN)
```

2.2 Check Details

Check	Method	Max Status	Threshold
Post-outcome name heuristic	Keyword scan on column names (default, defaulted, paid, outcome, label, etc.)	WARN	Any match
ID / proxy scan	n_unique / n_rows >= high_cardinality_ratio_threshold	WARN	0.90
Target correlation scan	Pearson |r| >= high_corr_threshold	WARN	0.85
Categorical proxy scan	max_rate - min_rate >= cat_target_rate_gap_threshold	WARN	0.65
Split distribution scan	Norm mean diff (numeric) >= threshold*3 TVD (categorical) >= threshold	WARN	0.35 / 1.05
Temporal availability	feature_ts > outcome_ts per row	FAIL	Any violation
Training future date scan	Auto-detected datetime cols vs max(outcome_ts) in train	FAIL	Any violation

2.3 The Two-FAIL Design

Only two checks produce FAIL: temporal availability and training future date scan. Both are temporal integrity violations -- the feature was computed after the outcome it is supposed to predict. These are the only findings with no legitimate alternative explanation. All other leakage signals (name heuristics, correlation, categorical proxy) can be explained by legitimate predictive features that happen to trigger the check. Only feature_ts > outcome_ts is structurally impossible in a valid prediction pipeline.

2.4 The LeakageAuditConfig Parameters

Parameter	Default	Controls
target_col	(required)	Name of the target/label column
split_col	None	Column with train/test labels; enables split distribution scan
outcome_time_col	None	Timestamp of the predicted event; enables temporal checks
feature_time_cols	{}	Dict mapping feature_name -> timestamp_col
high_corr_threshold	0.85	Pearson |r| threshold for target correlation WARN
categorical_target_rate_gap_threshold	0.65	Max-min target rate gap for categorical proxy WARN
high_cardinality_ratio_threshold	0.90	n_unique/n_rows for ID proxy WARN
distribution_shift_threshold	0.35	Base threshold for split distribution scan

Section 3 -- Algorithm Explanations: Math and Worked Examples

3.1 Pearson Correlation for Target Leakage (Check 3)

For each numeric feature, the check computes Pearson |r| against the target:

```
r(X, Y) = cov(X, Y) / (std(X) * std(Y)) = Σ[(Xi - X■)(Yi - ■)] / sqrt[Σ(Xi-X■)² * Σ(Yi-■)²]
```

The absolute value |r| is used because both positive and negative correlation can indicate leakage. If feature X is (1 - target), it has perfect negative correlation -- equally dangerous. The threshold of 0.85 is calibrated for production tabular datasets: legitimate predictive features rarely exceed |r| = 0.6 for binary targets in real-world financial, healthcare, or e-commerce data. |r| ≥ 0.85 is a strong signal of target encoding.

Worked example:

```
Feature: 'payment_received_flag' [0 if not received, 1 if received] Target: 'default' [1 if defaulted, 0 if not] If payment_received_flag = 1 whenever default = 0 and vice versa: r(payment_received_flag, default) ≈ -0.97 => |r| = 0.97 > 0.85 => WARN This feature is a near-perfect linear predictor of the target -- almost certainly a post-outcome column (payment received is recorded after the default/no-default decision).
```

Note on binary targets: for a binary {0,1} target, Pearson |r| is equivalent to the point-biserial correlation coefficient, which correctly measures the discrimination power of a continuous feature against a binary outcome. The implementation handles this correctly via numpy's corrcoef function.

3.2 Total Variation Distance for Categorical Split Shift (Check 5)

For categorical features, the split distribution scan uses Total Variation Distance (TVD) rather than mean difference (which only applies to numeric features):

```
TVD(P_train, P_test) = 0.5 * Σ_v |P_train(v) - P_test(v)| where P_train(v) = fraction of training rows with value v P_test(v) = fraction of test rows with value v
```

Worked example:

```
Feature: 'loan_region' Categories: ['North', 'South', 'East', 'West'] Train distribution: North=0.40, South=0.35, East=0.15, West=0.10 Test distribution: North=0.10, South=0.15, East=0.30, West=0.45 TVD = 0.5 * (|0.40-0.10| + |0.35-0.15| + |0.15-0.30| + |0.10-0.45|) = 0.5 * (0.30 + 0.20 + 0.15 + 0.35) = 0.5 * 1.00 = 0.50 > threshold (0.35) => WARN
```

3.3 Why TVD Over KL Divergence

KL Divergence (KL(P||Q) = Σ P(v) log(P(v)/Q(v))) has two problems for this use case: (1) KL divergence is undefined when P(v) > 0 but Q(v) = 0 -- a category present in training but absent in test causes a log(P/0) = infinity. This is common in tabular datasets with rare categories. (2) KL divergence is asymmetric: KL(P_train||P_test) ≠ KL(P_test||P_train). For a symmetric distribution shift check, asymmetry is unintuitive. TVD is bounded [0, 1], defined for all distributions (including zero probability), symmetric, and directly interpretable: TVD = 0.5 means half the distribution mass has shifted between train and test. These properties make it the correct choice for a practitioner-facing distribution shift metric.

3.4 Categorical Proxy Scan -- Target Rate Gap (Check 4)

For each categorical feature, compute the target positive rate within each category value:

```
rate(v) = mean(target[feature == v]) gap = max(rate) - min(rate) Threshold: gap >= 0.65 => WARN
```

Worked example:

```
Feature: 'loan_status' Target: 'default' (1=defaulted, 0=not) Values and target rates: 'active' : rate = 0.03 (3% of active loans default -- expected) 'in_review' : rate = 0.28 (28% -- elevated but plausible) 'written_off' : rate = 0.98 (98% -- almost all written-off loans defaulted) gap = 0.98 - 0.03 = 0.95 > 0.65 => WARN The 'written_off' category is a direct label proxy -- a loan is written off because it defaulted. This check catches what Pearson correlation cannot: categorical leakage.
```

3.5 Temporal Availability Check (Check 6)

For each row and each feature with a mapped timestamp column, compare the feature generation timestamp to the outcome timestamp:

```
For each row i: if feature_ts[i] > outcome_ts[i]: => FAIL (feature was computed AFTER the event we're predicting) Count the violations across all rows. Any violation = FAIL. Evidence: {future_rows: count, checked_rows: total_valid_rows}
```

3.6 Training Future Date Scan (Check 7)

This check auto-detects datetime-typed columns in the training split and checks whether any have values beyond the inferred training cutoff:

```
training_cutoff = max(outcome_ts) for training rows For each auto-detected datetime column col in train split: future_count = count(col[train] > training_cutoff) if future_count > 0: => FAIL Auto-detection: columns with datetime64 dtype OR string columns where >=80% of a 30-row sample parse as valid datetimes.
```

This catches temporal leakage that is invisible to the explicit temporal_availability check: features where no per-row timestamp mapping was provided, but whose values in the training set contain dates from the test period or beyond the training boundary. This was added as the 7th check in v0.4.0 because it fills a gap left by the temporal_availability check: you can't always provide a feature timestamp mapping, but you can always check whether a datetime column's values exceed the training cutoff.

3.7 Weighted Risk Score

v0.4.0 added a weighted_leakage_risk_score that computes a scalar summary of finding severity across all checks:

```
Weights: FAIL / high = 4.5 FAIL / medium = 3.0 FAIL / low = 1.5 WARN / high = 1.5 WARN / medium = 1.0 WARN / low = 0.3 risk_score = sum(weight[status, severity] for each finding) Example: 1 FAIL/high + 3 WARN/medium = 4.5 + 3.0 = 7.5 Interpretation: higher score = more dangerous feature set.
```

Section 4 -- Design Decisions: Why Things Are the Way They Are

Why WARN for Most Checks, FAIL Only for Temporal

Most leakage signals are probabilistic, not deterministic. A feature named 'resolved_date' might be a post-outcome feature, or it might be a pre-outcome resolution step in the business process. A feature with |r| = 0.85 against the target might be genuine (an exceptionally predictive legitimate feature) or leaked. Only temporal availability -- feature_ts > outcome_ts -- has no legitimate explanation. A feature whose generation timestamp is provably after the outcome timestamp cannot have been available at prediction time. This is a structural impossibility, not a suspicious pattern. The WARN/FAIL distinction matters for adoption: a tool that flags everything as FAIL creates alarm fatigue and gets ignored. A tool that fires FAIL only on genuine structural violations gets taken seriously.

Why No Automatic Feature Removal

FeatureLeakageLens produces a report; it does not modify the DataFrame. This is intentional. Automated feature removal based on leakage heuristics would silently delete legitimate features that happen to trigger the checks (false positives). A high-correlation feature might be genuinely predictive. A feature with a post-outcome-sounding name might have been renamed but is actually pre-outcome. Every finding requires a human decision: review the finding, determine whether it represents actual leakage, and remove the feature manually if confirmed. This keeps the human in the loop for irreversible data decisions.

Why Two FAIL Checks Instead of One

The original design (v0.1-v0.3) had only temporal_availability as a FAIL check. The training_future_date_scan was added in v0.4.0 because there's a common pattern the first check misses: date columns in the training data whose values extend into the future relative to the training boundary, even when no per-row timestamp mapping was provided by the user. A rolling 7-day average computed post-split, a 'last_activity_date' that was joined from a post-cutoff data snapshot -- these don't get caught by temporal_availability (no explicit feature-to-timestamp mapping provided) but get caught by the automatic datetime detection in training_future_date_scan.

Why Zero External Dependencies (Beyond NumPy and Pandas)

The core audit logic (LeakageAuditor) has no external dependencies beyond NumPy and pandas. This is deliberate. Most ML pipelines already have NumPy and pandas in their environment. Adding scikit-learn, statsmodels, or other statistical libraries would create dependency conflicts in environments with specific version pinning. The zero-extra-dependency design means pip install featureleakagelens works in any ML environment without conflict. Optional HTML output uses only Python's html module from the standard library.

Why INSUFFICIENT_INPUT is Not PASS for Split Distribution Scan

If no split_col is provided, the split distribution scan cannot run. It returns INSUFFICIENT_INPUT rather than PASS. This distinction matters: 'no split data provided' is not the same as 'no distribution shift found.' A downstream consumer reading the report should know that this check was not evaluable, not that it passed. This same principle applies to temporal checks without timestamps: INSUFFICIENT_INPUT explicitly signals an audit gap.

Why the Truth Boundary Is Mandatory and Cannot Be Suppressed

FeatureLeakageLens flags suspicious patterns. It does not prove leakage. Every finding requires domain expert confirmation. Only temporal availability (feature_ts > outcome_ts) is unambiguous. The truth boundary text is mandatory and included in every output format. It cannot be suppressed from JSON, Markdown, or HTML output. The reason: reports that omit the interpretation caveat get used as 'proof of no leakage', which is incorrect. Attaching the truth boundary text to every output format makes it impossible to share the report without the disclaimer.

Section 5 -- Honest Positioning: What Is and Is Not Built

> Precision about what is implemented vs. planned is a signal of engineering maturity. The interviewer is not looking for a complete product -- they are looking for a practitioner who understands the difference between what they built and what they claim to have built.

[IMPLEMENTED]	All 7 checks run on real DataFrame input
[IMPLEMENTED]	PASS/WARN/FAIL/INSUFFICIENT_INPUT status hierarchy
[IMPLEMENTED]	JSON, Markdown, HTML output formats with per-finding evidence
[IMPLEMENTED]	CLI with exit codes 0/1/2/3
[IMPLEMENTED]	PyPI published (pip install featureleakagelens)
[IMPLEMENTED]	Weighted risk score per-finding (FAIL/high=4.5, WARN/medium=1.0, etc.)
[IMPLEMENTED]	Mandatory truth boundary on all output formats
[IMPLEMENTED]	LeakageAuditConfig with all thresholds configurable
[IMPLEMENTED]	Pearson |r| for numeric target correlation
[IMPLEMENTED]	TVD for categorical split distribution shift
[IMPLEMENTED]	Auto-detection of datetime columns for training_future_date_scan
[IMPLEMENTED]	Weighted leakage risk score aggregation
[SIMULATED]	Demo dataset (demo_leakage_dataset.csv) with deliberate leakage injected
[SIMULATED]	generate_demo_reports.py demonstrating all 7 check types
[FUTURE]	Mutual information scan for nonlinear proxy detection
[FUTURE]	Group leakage check: same entity in train and test (cross-val leakage)
[FUTURE]	Time-series walk-forward split validator
[FUTURE]	Text/NLP feature leakage: label-in-text detection for NLP features
[FUTURE]	Feature interaction leakage: product of two legitimate features encoding target
[FUTURE]	Auxiliary table join audit: features joined after the train/test split

Section 6 -- Q&A; Battery (30+ Pairs)

These questions map to what FAANG ML engineers ask when they probe a data quality tool. The answers explain first principles -- not just what the check does, but why that design decision was made.

Q1: Walk me through what happens when a feature has perfect correlation with the target -- is that always leakage?

Not always, but it is always worth investigating. Perfect or near-perfect Pearson |r| with the target is one of the strongest signals of leakage, but there is one legitimate scenario where it occurs: a single feature that is genuinely the best predictor in the domain. In medical diagnostics, a specific biomarker can have $|r| \geq 0.90$ with a binary diagnosis. In credit, a single bureau score can approach $|r| = 0.80$ with default. FeatureLeakageLens flags these as WARN -- not FAIL -- specifically because the finding requires domain expert review. The question to ask is: 'Was this feature generated using information that would not be available at prediction time?' If the answer is definitively yes (feature timestamp > outcome timestamp), it's FAIL. If the answer is uncertain, it's WARN. The tool doesn't pretend to know which case it is -- the human domain expert confirms.

Q2: What's the difference between target leakage and temporal leakage?

Target leakage is the broader category: any feature that encodes information about the target value directly or indirectly. It includes: post-outcome columns (computed after the event), label proxies (categorical values that directly encode the outcome), and ID leakage (high-cardinality IDs that correlate with target through a confounder). Temporal leakage is a specific type of target leakage where the mechanism is time-based: the feature was computed using data from after the prediction time. A rolling average that looks forward in time is temporal leakage. A post-outcome column is also temporal leakage -- it was generated after the event. The distinction matters for detection: temporal leakage can be detected mechanically using timestamps (feature_ts > outcome_ts = FAIL). Target leakage without a temporal signal is harder to detect -- it requires checking name heuristics, correlation, and categorical proxy patterns, all of which produce WARN, not FAIL.

Q3: Your temporal check looks at feature creation timestamps -- what if timestamps are missing?

If no outcome_time_col or feature_time_cols are provided, the temporal_availability check returns INSUFFICIENT_INPUT -- explicitly not PASS. This is documented in the report and signals an audit gap: the most reliable leakage detection method was not evaluable. The training_future_date_scan (Check 7) was designed specifically for this scenario: it auto-detects datetime-typed columns in the training split and checks whether any contain values beyond the inferred training cutoff (max(outcome_ts) for training rows). This catches a common pattern: a feature column whose values are dates, and some of those dates fall after the training boundary, even when no explicit per-row timestamp mapping was provided. If both checks return INSUFFICIENT_INPUT (no outcome timestamp provided at all), the report says so explicitly: temporal audit was not possible. This is the honest answer -- the tool doesn't pretend it checked when it didn't.

Q4: How does the ID proxy scan work -- what makes something ID-like?

The ID proxy scan uses two signals: naming pattern and cardinality. Naming: the column name ends in 'id', ends in '_id', or contains 'identifier' (case-insensitive). Cardinality: n_unique(column) / n_rows >= high_cardinality_ratio_threshold (default 0.90). A column is flagged WARN if it matches the naming pattern AND exceeds the cardinality threshold. Why both signals? A column named 'customer_id' with only 5 unique values across 10K rows is not an ID-like leakage risk -- it's probably a segment or group code. A high-cardinality column named 'zip_code' is high-cardinality but not obviously ID-like (no id-naming pattern). The concern with ID columns: if an ID was assigned post-outcome (e.g., a claim ID assigned after a fraud decision), it encodes the outcome. Or if IDs are sequential and the data was collected across periods with different label distributions, the ID range encodes the temporal cohort -- indirect temporal leakage.

Q5: Pearson correlation detects linear leakage -- what about non-linear leakage?

Pearson correlation misses non-linear leakage entirely. A feature that is the square of the target (X = target^2) has near-zero Pearson correlation with the target (for a zero-mean target), but is a perfect predictor. A categorical feature that encodes the target through a non-monotonic mapping is also invisible to Pearson. FeatureLeakageLens addresses this partially through the categorical proxy scan: if a categorical feature's values have a 65%+ target rate gap, it's flagged regardless of correlation. This catches the non-linear categorical case. However, non-linear continuous features (X = target^2, X = abs(target), polynomial relationships) are not caught by any current check. This is a documented limitation in the truth boundary. The FUTURE item is a mutual information scan: I(X; target) computed via k-nearest-neighbor estimation, which captures any functional relationship between the feature and target, linear or not.

Q6: What's Total Variation Distance and why use it over KL divergence for the categorical proxy scan?

Wait -- clarification: TVD is used for the split distribution scan (Check 5), not the categorical proxy scan. The categorical proxy scan (Check 4) uses the target rate gap: max_rate - min_rate across category values. For split distribution (Check 5): TVD(P_train, P_test) = 0.5 * Σ |P_train(v) - P_test(v)|. TVD is preferred over KL divergence for two reasons: (1) KL divergence is undefined when P(v) > 0 but Q(v) = 0 -- a category present in training but absent in test causes log(P/0) = infinity. Tabular datasets routinely have categories present in only one split (rare values, sampling effects). TVD handles zero probabilities gracefully: the contribution is just |P(v) - 0| = P(v). (2) TVD is bounded [0, 1] and directly interpretable: TVD = 0.5 means that 50% of the probability mass has shifted between train and test. KL divergence is unbounded and not interpretable as a fraction. For a practitioner-facing threshold metric, boundedness and interpretability matter.

Q7: What's the training_future_date_scan and why was it the 7th check added last?

The training_future_date_scan was added in v0.4.0 as the 7th check. It auto-detects datetime-typed columns in the training split and checks whether any values exceed the inferred training cutoff (max(outcome_ts) in the training rows). Any training row with a feature datetime after the training cutoff is a temporal leakage violation: FAIL. It was added last because it fills a gap that emerged from real usage of the first 6 checks: teams would correctly configure outcome_time_col but not realize they needed to enumerate every datetime feature in feature_time_cols. Features like 'last_login_date', 'last_transaction_date', 'contract_renewal_date' would slip through the temporal_availability check because no timestamp mapping was provided for them. The auto-detection approach -- scan for datetime-typed columns, check against training cutoff -- catches this without requiring the user to enumerate every datetime feature. It was designed to require minimal configuration: it activates whenever split_col and outcome_time_col are provided, with no additional input needed.

Q8: How would you handle leakage in a time-series cross-validation setting?

FeatureLeakageLens in its current form is designed for a single train/test split. Time-series cross-validation (expanding window or rolling window) creates k folds, each with a different train/test boundary. The current tool would need to be run k times -- once per fold -- to audit each fold's training data independently. The more important concern in time-series CV is group leakage: if the same entity (customer, store, asset) appears in both the training and test folds of a given CV split, features like historical averages for that entity encode test-period information. This is not caught by any current check -- it requires knowing the entity identifier and verifying that no entity appears in both train and test. This is the top FUTURE item on the roadmap: a group leakage check for cross-validation folds. The implementation would take a fold_col and entity_col, then flag entity IDs that appear in both train and test for a given fold.

Q9: What does WARN vs FAIL mean -- why do you never FAIL on leakage?

The design principle: FAIL is reserved for findings where there is no legitimate alternative explanation. For temporal violations (feature_ts > outcome_ts), there is no scenario in a valid prediction pipeline where the feature was generated after the event it predicts -- unless the feature is being misused. This gets FAIL. For everything else -- name heuristics, correlation, categorical proxy, distribution shift -- there is a legitimate interpretation: the feature might just be highly predictive without being leaked. The tool cannot distinguish between 'this feature has |r| = 0.87 because it's a genuine predictor' and 'this feature has |r| = 0.87 because it encodes the label through a post-outcome proxy.' Only a domain expert with knowledge of the feature's generation process can make that determination. Firing FAIL on these patterns would block model training on legitimate high-signal features, creating exactly the alarm fatigue that causes teams to disable the tool.

Q10: How is this different from running feature importance and looking at suspiciously high scores?

Feature importance from a trained model has a fundamental problem: you've already trained the model on the leaky features. The importance scores reflect model learning, not leakage detection. A leaky feature will have high importance -- but so will a genuinely predictive legitimate feature. You can't distinguish them from importance alone. FeatureLeakageLens runs before any model is trained. The checks are independent of the model: they examine the feature values, timestamps, and distributions directly. This means leakage is detected before it corrupts the model, not after. Additionally, feature importance misses specific patterns: it won't flag a categorical feature as leaky if it happens to have lower tree-split importance than numeric features. The categorical proxy scan specifically targets this: it checks target rate gaps regardless of importance score. The two approaches are complementary: FeatureLeakageLens pre-training + post-training feature importance review covers different failure modes.

Q11: What's your false negative rate -- what kinds of leakage does this miss?

FeatureLeakageLens misses several important leakage patterns: (1) Nonlinear continuous leakage: X = target^2, or any non-monotonic relationship between a continuous feature and the target. Pearson |r| will not catch this. (2) Leakage through feature interactions: if feature A and feature B are each individually legitimate but their product (A*B) or ratio encodes the target, no current check catches this. (3) Label-in-text leakage: a description field containing the word 'defaulted' or 'approved'. The name heuristic only checks column names, not column values. (4) Auxiliary table join leakage: features joined from a table that was created after the training cutoff. The tool has no visibility into how features were constructed -- only their final values and timestamps. (5) Rolling aggregation leakage: a 7-day rolling average computed on the full dataset before the split -- future-peeking in windowed features. Detectable only if the computation timestamp is recorded. This is why the truth boundary is mandatory: every PASS finding means 'no suspicious pattern detected by these 7 checks', not 'no leakage exists'.

Q12: How would this scale to 10,000-feature datasets?

The 7 checks scale differently with feature count: Name heuristic (Check 1): O(k) where k = feature count. Scales linearly, trivial. ID scan (Check 2): O(n*k) -- one nunique() call per feature over n rows. At 10K features and 1M rows: ~10 seconds. Target correlation (Check 3): O(n*k) -- one Pearson correlation per feature. Vectorized with numpy.corrcoef for all features simultaneously: O(n*k) total, ~5 seconds at 10K features, 1M rows with float64 data. Categorical proxy (Check 4): O(n*k) worst case, but skips features with high cardinality (above max_unique_ratio_for_categorical_scan). At 10K features, most will be numeric and skipped. Split distribution (Check 5): O(n*k). Temporal (Checks 6-7): O(n) regardless of feature count -- operates on timestamp columns only. Total at 10K features, 1M rows: ~30-60 seconds on a modern laptop. Memory: 1M rows * 10K float64 features = 80 GB -- this is the actual bottleneck. At this scale, the tool would need to operate on chunked DataFrames rather than loading everything into memory. This is a FUTURE architecture item.

Q13: What's the most common leakage pattern you'd catch that others miss?

The categorical proxy scan. Teams routinely check numeric correlations but don't systematically check categorical target rates. A loan_status_category column with values like 'active', 'pending_review', and 'written_off' will have a target rate of 0% for active, 15% for pending review, and 100% for written_off. The 'written_off' category is a perfect target proxy -- completely invisible to Pearson correlation (it's not numeric) and invisible to name heuristics (the column name sounds legitimate). The categorical proxy scan catches this by computing target rate per category value and flagging the 100 - 0 = 100% gap (>> 65% threshold). This is the leakage pattern that ships most often in financial services ML because categorical status fields are a natural part of the data model and feel like legitimate features until you compute the per-value default rates.

Q14: How does the tool handle high-cardinality categoricals like zip codes?

The categorical proxy scan has a cardinality gate: features with unique ratio > max_unique_ratio_for_categorical_scan (default: 0.25) are skipped for the target rate computation. A zip code column with 40,000 unique values in 100,000 rows (ratio = 0.40) is above this threshold and skipped. The reasoning: for very high-cardinality categoricals, per-value target rates are estimated from very few rows per value. A zip code with 3 rows, all defaulting, would show a 100% target rate -- but this is a small-sample artifact, not leakage. The check would produce too many false positives to be useful. The trade-off is that the check may miss real leakage in high-cardinality categorical columns where leakage is concentrated in specific high-frequency values. For cases where you want to check high-cardinality features, set max_unique_ratio_for_categorical_scan to a higher value and increase the minimum per-value sample count (the code filters values with fewer than 5 rows).

Q15: What happens if you call it without a split_col?

The split distribution scan (Check 5) returns INSUFFICIENT_INPUT instead of PASS. This distinction is documented in the report and is intentional: 'no split data provided' is not the same as 'no distribution shift found.' The training_future_date_scan (Check 7) also returns INSUFFICIENT_INPUT because it requires both split_col and outcome_time_col to identify training rows and infer the training cutoff. The overall status: if no checks FAIL or WARN, the tool returns PASS even with INSUFFICIENT_INPUT checks. The INSUFFICIENT_INPUT findings appear in the findings list with their recommendation ('Provide split_col to enable distribution shift audit'). This is the key behavior: INSUFFICIENT_INPUT appears in the findings but does not escalate the overall status to INSUFFICIENT_INPUT -- it provides information about what was not audited without blocking the user.

Q16: How does FeatureLeakageLens fit into a CI/CD ML pipeline?

The CLI exit code contract maps directly to CI/CD pipeline control: PASS (0): all checks passed, model training can proceed. WARN (1): suspicious patterns found, model training can proceed with documented review. FAIL (2): temporal violation confirmed, model training is blocked. INSUFFICIENT_INPUT (3): some checks could not run -- informational. In a GitHub Actions pipeline: - steps.audit.exit_code == 2 fails the step and blocks downstream model training. - steps.audit.exit_code == 1 triggers a PR comment with the WARN findings, requiring a human override comment before training is unblocked. The structured JSON output (to_json()) can be fed to a PR comment template that formats the findings as a table. The HTML report can be uploaded as a workflow artifact for review.

Q17: Walk me through the post_outcome_name_heuristic check in detail

The check scans column names for a list of post-outcome keyword terms: default, defaulted, paid, payment_received, settled, recovered, approved, approval, denied, closed, chargeback, refund, outcome, target, label, bad_flag, delinquent, fraud_confirmed. For each feature column, the check lowercases the column name and checks whether any term appears as a substring. If matched, the column is flagged WARN with the matched terms as evidence. This check is zero-compute -- it runs on column names alone before any row of data is read. It is intentionally low-precision (high false positive rate on legitimate columns like 'payment_method' triggering on 'payment') because the cost of a false negative (missing a genuinely leaky column) vastly exceeds the cost of a false positive (a human reviews a suspicious-sounding column name). The check trades precision for recall: flag everything suspicious, let the human confirm.

Q18: What's the version history and what did each version add?

v0.1.0: first four checks -- name heuristic, ID scan, target correlation, categorical proxy. JSON and Markdown output. Basic CLI. v0.2.0: added split distribution scan (Check 5). HTML output format. Color-coded report (FAIL=red, WARN=amber, PASS=green). v0.3.0: expanded categorical proxy scan to handle high-cardinality categoricals (>50 unique values) using sampling-based target rate estimation. Prevents timeouts on zip code / product ID columns. v0.4.0 (current): added temporal boundary scan (training_future_date_scan, Check 7) that auto-detects datetime columns vs. training cutoff. Added weighted risk score (per-finding severity weight, aggregated to scalar 0-10+). The 7th check was added because users were not consistently mapping all datetime features in feature_time_cols, leaving a detection gap.

Q19: How does FeatureLeakageLens connect to RiskFrame in the portfolio?

RiskFrame is the portfolio's ML model lifecycle platform -- it handles champion/challenger model management, drift monitoring, and fairness checks. FeatureLeakageLens is the pre-training gate that runs before any model in RiskFrame is trained. Specifically: before training the XGBoost champion on Home Credit data in RiskFrame, FeatureLeakageLens audits the feature set for target leakage (features that encode default status directly, like post-default payment flags), temporal leakage (features computed after the loan decision date), and near-duplicate features. The credit_income_ratio engineered feature passes all checks -- it is a pre-decision ratio, not a post-outcome signal. This is documented in the RiskFrame model card. If FeatureLeakageLens had not been run and a leaky feature made it into the XGBoost champion, RiskFrame's drift monitoring would eventually catch the production performance collapse -- but by then, decisions have already been made on a compromised model.

Q20: What's the difference between the split_distribution_scan WARN and the categorical_proxy_scan WARN?

Split distribution scan (Check 5): compares the distribution of a feature between train and test. It asks: 'Does this feature have a different distribution in the test set compared to training?' A large shift suggests the split is not representative or the feature has different statistics in different time periods -- which can indicate temporal leakage or data collection artifacts. Categorical proxy scan (Check 4): examines the distribution of the target variable across the feature's values. It asks: 'Does this feature predict the target too well within a single split?' A 65%+ target rate gap across category values suggests the feature directly encodes outcome state. They're orthogonal: a feature can WARN on both (high train/test shift AND high categorical target rate gap), one but not the other, or neither. The split distribution scan doesn't need a leaky feature -- it detects shift. The categorical proxy scan doesn't care about splits -- it detects target encoding.

Q21: How would you debug a FAIL from temporal_availability in practice?

Step 1: Look at the evidence in the finding: how many future_rows, what fraction of checked_rows. If future_rows = 3 out of 50,000, it may be a data quality issue (late-arriving records, timestamp rounding errors) rather than systematic leakage. If future_rows = 15,000 out of 50,000, it's systematic and must be fixed. Step 2: Examine the specific rows where feature_ts > outcome_ts. What is the feature value for those rows? Is the timestamp consistently a fixed offset from the outcome timestamp (e.g., always 1 day later)? That suggests the feature generation process is using the outcome timestamp as a reference. Step 3: Trace the feature generation code. How was the timestamp column produced? If it's a 'last_updated' timestamp from a database that was captured after the outcome was recorded, the feature may still be valid if the underlying feature value was computed before the outcome -- the timestamp is just when the record was written. Step 4: If the feature is confirmed leaky, remove it from the feature set and re-run the audit. If it's a data quality issue (late timestamps on legitimate records), clip the feature timestamp at the outcome timestamp for those rows and document the fix.

Q22: What happens if the target column is continuous (regression) instead of binary (classification)?

The name heuristic (Check 1) and ID scan (Check 2) are label-type agnostic -- they work the same for regression. Target correlation (Check 3): Pearson |r| for regression targets works correctly -- it measures linear correlation between the feature and the continuous outcome. Categorical proxy scan (Check 4): this check requires a binary or near-binary target (y.nunique() == 2). For continuous targets, the check skips the target rate gap computation. This is a limitation: categorical proxies for continuous targets (e.g., a category value that predicts very high vs. very low target values) are not currently caught. Temporal checks (Checks 6-7): fully agnostic to label type. Split distribution (Check 5): works for both -- numeric shift uses standardized mean difference, categorical TVD is target-agnostic. The practical recommendation for regression: the correlation threshold (0.85) may need to be adjusted -- legitimate regression features can have higher correlations with continuous targets than with binary targets.

Q23: How is this different from pandas-profiling or ydata-profiling?

pandas-profiling / ydata-profiling generates descriptive statistics and visualizations for each column: missing value rates, distribution histograms, correlation heatmaps, and high-correlation warnings. It is a data exploration tool, not a leakage auditor. It does not: check whether a feature was generated after the outcome, compute categorical target rate gaps (the tool would need to know which column is the target), detect ID proxy leakage through cardinality + naming patterns, or produce PASS/WARN/FAIL status with a machine-readable exit code for CI. FeatureLeakageLens is domain-specific (ML feature leakage) and production-ready (CLI, exit codes, CI-attachable output). pandas-profiling is general-purpose exploration. The two tools serve different moments in the workflow: profiling at initial data exploration, FeatureLeakageLens at pre-training model card review.

Q24: What's the weighted risk score and how should practitioners interpret it?

The weighted risk score is a scalar that aggregates the severity of all findings: FAIL/high = 4.5 points, FAIL/medium = 3.0, FAIL/low = 1.5, WARN/high = 1.5, WARN/medium = 1.0, WARN/low = 0.3. risk_score = sum of weights across all findings. Interpretation: a dataset with 0 findings has risk_score = 0. A dataset with 1 temporal FAIL/high has risk_score = 4.5 -- stop, fix the leakage. A dataset with 4 name heuristic WARNs (each medium) has risk_score = 4.0 -- review the column names, likely some false positives. There is no universal threshold for 'safe' -- the risk score is a ranking tool for feature prioritization, not a binary gate (that's what the FAIL/WARN status is for). Teams can use it to prioritize which features to review first in large feature sets: sort by finding risk_score descending and review top-10.

Q25: How would you test this tool's detection accuracy?

Three-part test strategy: (1) Synthetic injection tests: create a DataFrame with known-good features, inject a specific leakage pattern (post-outcome column, temporal violation, categorical proxy), assert the corresponding check fires FAIL or WARN, and assert the evidence fields match the injected values. This is the core of the 23-test suite. (2) False positive tests: create a DataFrame with features that superficially resemble leaky patterns but are legitimate (a genuinely predictive high-correlation feature, a short column name containing 'outcome' that is actually a business input), assert the check fires WARN (not FAIL), and verify the finding is reviewable rather than a blocker. (3) End-to-end regression test: the demo dataset with deliberate leakage injections is run through generate_demo_reports.py, and the output report is compared against a known-good reference output. Any change to check logic that changes the demo report fails this test.

Q26: Explain the INSUFFICIENT_INPUT / PASS distinction for split_distribution_scan

When split_col is not provided: split_distribution_scan returns INSUFFICIENT_INPUT and adds a finding with recommendation 'Provide split_col to audit train/test distribution warnings.' When split_col is provided but the column is missing from the DataFrame: INSUFFICIENT_INPUT with a different message ('Split column was configured but missing'). When split_col is valid but either split has fewer than 10 rows: INSUFFICIENT_INPUT with message about insufficient sample size. When split_col is valid and splits are adequate: the check runs and returns PASS (no shift detected) or WARN (shift above threshold). The distinction matters: a model card that says 'split distribution: PASS' has a very different meaning than one that says 'split distribution: INSUFFICIENT_INPUT'. The first means no problematic shift was found. The second means the check was never run. Both situations are documented explicitly in the report.

Q27: What's the most dangerous leakage pattern FeatureLeakageLens does NOT catch?

Rolling aggregation leakage -- a feature computed as a rolling window average over the full dataset before the train/test split. For example: a 30-day rolling average of customer purchases computed on the entire date range, then used as a feature in a model where the test set is the last 30 days. The rolling average for test-period rows includes purchases that happen after those rows in time -- future-peeking. This leakage pattern requires knowledge of how the feature was computed, not just what its values and timestamps are. FeatureLeakageLens operates on the final feature values -- it cannot inspect the feature engineering code. The only way to detect this is: (1) per-row feature generation timestamps that record when the rolling window was computed (caught by temporal_availability), or (2) reviewing the feature engineering code directly (human audit, not automated). This is why the truth boundary is non-negotiable: the tool cannot catch what it cannot see.

Q28: If you had to pick the single most important check, which one would it be?

Temporal availability (Check 6) -- the only FAIL check with a completely unambiguous finding. When feature_ts > outcome_ts for any row, there is no legitimate explanation in a valid prediction pipeline. The feature was generated after the event it predicts. This is not a heuristic, not a threshold, not a probabilistic signal. It is a structural impossibility that produces a deterministic FAIL. All other checks produce probabilistic WARNs that require human review. The temporal check produces a definitive answer. If I had to run one check and only one, temporal availability catches the worst case: a model that would have AUC = 0.99 in development and collapse to AUC = 0.52 in production. The training_future_date_scan (Check 7) is a close second -- it covers the temporal check's blind spot (no explicit timestamp mapping provided) with auto-detection.

Q29: How does the tool's output attach to a model card?

The JSON output (to_json()) produces a structured payload that can be included in a model card's data section: overall status, per-finding details with evidence, summary counts (warn_count, fail_count), and the truth boundary text. The recommended model card entry: - Pre-training leakage audit: FeatureLeakageLens v0.4.0 - Status: WARN (2 findings) - Findings: payment_received_flag (name heuristic match), loan_id (ID proxy) - Review: both confirmed legitimate by domain expert on 2025-03-14 - Truth boundary: [mandatory text included] The HTML report can be stored as an artifact linked from the model card. The Markdown output can be directly embedded in a README or PR description. This is the intended workflow: audit before training, attach findings to model card, document domain expert review decisions.

Q30: What would v1.0 add?

1. Mutual information scan for nonlinear proxy detection: I(X; target) via k-nearest-neighbor estimation, capturing any functional relationship between feature and target, not just linear (Pearson) relationships. 2. Group leakage check: for cross-validation, flag entity IDs appearing in both train and test folds. Takes entity_col and fold_col as configuration. 3. Text feature leakage: for NLP features, scan the feature text values for occurrences of target-associated vocabulary (e.g., the word 'defaulted' appearing in a loan description field). 4. Rolling aggregation audit: flag features whose names match window/rolling patterns ('7d_avg', 'rolling_mean_30', 'cumsum') and require proof that the window was computed point-in-time. 5. Feature interaction audit: for top-k features (by risk score), compute pairwise products/ratios and check their correlation with the target -- catching two legitimate features whose interaction encodes the label.

Q31: Why are there no WARN-to-FAIL promotions in the config?

The current design does not support promoting WARN checks to FAIL through AuditConfig. This is a deliberate conservative choice: the 7 checks were designed with their FAIL/WARN levels reflecting the confidence of the finding. Allowing configuration-based FAIL promotions would enable teams to misconfigure the tool into blocking model training on uncertain findings (e.g., FAIL on any name heuristic match would block training whenever a column name contains 'default', even if it's a legitimate feature). A FUTURE configuration option -- fail_on_warn_checks: list[str] -- would allow teams with strict compliance requirements to promote specific WARN checks to FAIL, with the expectation that they understand the false positive implications and have pre-reviewed their feature names against the keyword list.

Q32: How does FeatureLeakageLens connect to DevPulse in the portfolio?

DevPulse is the portfolio's version-safe RAG platform. Its training data for the version classification model -- which determines whether retrieved documents are from the correct product version -- is audited by FeatureLeakageLens to ensure version labels don't appear in the feature text itself. This is label contamination in the NLP context: a description field mentioning 'version 4.2' when the label is 'v4.2' would give the classifier trivial text-matching ability that collapses in production when version references are not present. FeatureLeakageLens flags text features for manual review through the name heuristic (if the column is named 'version_text' or 'label_description') and through the truth boundary note that explicitly calls out NLP label contamination as a pattern outside the current detection scope.

Section 7 -- Failure Modes: What the Auditor Misses

> The mark of an engineer who actually built and used a tool is their ability to articulate exactly where it breaks. These are the honest limitations of FeatureLeakageLens.

Nonlinear Leakage

Pearson correlation is a linear measure. X = target^2, X = sin(target), and any non-monotonic transformation of the target have near-zero |r| with the target. A feature that is 1 when target > median(target) and 0 otherwise has |r| ≈ 0 (it's the step function of the target, not a linear proxy). Mutual information (FUTURE) would catch this.

Feature Interaction Leakage

Two individually clean features whose product or ratio encodes the target. Example: feature A = loan_amount, feature B = monthly_income. Both are legitimate. Feature A/B = debt-to-income ratio, which is strongly predictive. But if one data scientist adds A/B as an engineered feature and another adds A*B that happens to be correlated with a post-outcome metric, the interaction check is needed. Not currently implemented.

Text/NLP Label Contamination

The name heuristic checks column names. It does not check column values. A 'loan_notes' text column containing 'Customer has been contacted for default resolution' is invisible to all 7 checks. The word 'default' in the column value is not caught. NLP feature leakage requires value-level scanning, which is a FUTURE item.

Auxiliary Table Join Leakage

Features joined from a table that was captured after the training cutoff. The tool sees the final feature values -- it cannot see how they were produced. A 'bureau_score' column joined from a credit bureau table that was last refreshed after the training period would look like a legitimate feature.

Threshold Miscalibration for Specific Domains

The default high_corr_threshold=0.85 is calibrated for financial/credit datasets. In other domains: medical diagnostics (where |r| = 0.90 can be legitimate), manufacturing quality control (almost deterministic relationships are common), or A/B testing (where small effect sizes make |r| = 0.85 implausible). Teams should validate thresholds on known-good datasets before using as a CI gate.

False Positive Flood from Name Heuristics

The POST_OUTCOME_TERMS list includes 'approved', 'closed', 'paid', 'target', 'label'. In datasets with domain-standard naming, these terms appear in legitimate feature names: 'pre_approved_amount' (not post-outcome), 'closed_account_count' (a pre-decision metric), 'payment_type_label' (a feature category label,

not the target). High false positive rates from name heuristics can cause teams to ignore WARNs entirely -- which defeats the purpose.

Section 8 -- Scalability: 10K Features / 10M Rows

8.1 Complexity by Check

Check	Complexity	100K rows, 500 features	1M rows, 10K features
Name heuristic	O(k)	Instant	Instant
ID scan	O(n*k)	~0.5s	~50s (vectorizable)
Target correlation	O(n*k)	~1s (numpy vectorized)	~30s
Categorical proxy	O(n*k')	~2s (k' << k)	~20s
Split distribution	O(n*k)	~1s	~40s
Temporal availability	O(n)	~0.1s	~1s
Training future date	O(n*d)	~0.5s	~5s
TOTAL	O(n*k)	~5s	~3 min

8.2 Memory Constraints

Dataset	Memory (float64)	Feasibility
100K rows, 100 features	~80 MB	In-memory, no issue
1M rows, 500 features	~4 GB	Requires adequate RAM
1M rows, 10K features	~80 GB	Chunked processing required (FUTURE)
10M rows, 100 features	~8 GB	Feasible with 16 GB RAM

8.3 Optimization Opportunities

The target correlation scan currently runs one Pearson correlation per feature. This can be vectorized: compute all correlations simultaneously using numpy.corrcoef(df[candidate_cols].values.T, target.values) -- returns the full correlation matrix in one BLAS operation, roughly 10-50x faster than looping. The categorical proxy scan has a natural cardinality gate (skips high-cardinality features) that limits its scope. For 10M+ row datasets, sampling is the correct approach: run checks on a stratified sample of 100K-500K rows. For temporal checks (FAIL-level), always run on the full dataset -- a temporal violation in the 5% of rows not in the sample must not be missed.

Section 9 -- Cross-Project Connections

9.1 The Pre-Training Data Quality Gate

FeatureLeakageLens is the pre-training data quality gate for all supervised ML projects in this portfolio. Its position in the development workflow:

```
Data Collection / Feature Engineering | v [FeatureLeakageLens Audit] <-- HERE FAIL --> Stop. Fix temporal violation before training. WARN --> Review findings. Document domain expert decision. PASS --> Proceed to model training. | v Model Training (XGBoost, LightGBM, neural network) | v [RiskFrame: Champion/Challenger / Drift Monitoring]
```

9.2 Connection to RiskFrame (ML Model Lifecycle Platform)

RiskFrame manages the ML model lifecycle: champion/challenger model management, drift monitoring, and fairness checks. FeatureLeakageLens gates the entry to RiskFrame's training phase. A model that enters RiskFrame with clean features (no FAIL, reviewed WARNs) has a trustworthy AUC baseline. A model that enters with leaky features has an AUC baseline that is an artifact, not a measurement. When RiskFrame's drift monitoring detects a sudden production performance drop, the first question is: 'Was FeatureLeakageLens run before this model's training?' If not, feature leakage is the first hypothesis.

9.3 Connection to DocIngestQA and GoldenSetAuditor

All three tools are audit/quality-gate tools in the portfolio, each at a different stage of the ML and AI pipeline:

Tool	When	What It Gates
DocIngestQA	Pre-indexing (RAG)	Chunk structural quality
FeatureLeakageLens	Pre-training (tabular ML)	Feature leakage patterns
GoldenSetAuditor	Pre-evaluation (RAG/LLM)	Eval dataset quality

Section 10 -- Resume Claim Audit

Resume Claim (Exact Text):

> "Built FeatureLeakageLens, a pre-training feature leakage auditor for tabular ML datasets that checks for post-outcome name heuristics, target correlation, categorical target-rate proxies, future timestamp leakage, ID/proxy columns, and train/test distribution shift, producing structured JSON/Markdown/HTML audit reports with per-finding PASS/WARN/FAIL/INSUFFICIENT_INPUT status and explicit truth boundary."

Claim	Status	Evidence
7 checks	[IMPLEMENTED]	All 7 checks in core.py, all tested
Post-outcome name heuristics	[IMPLEMENTED]	POST_OUTCOME_TERMS keyword scan on column names
Target correlation	[IMPLEMENTED]	Pearson |r| via numpy.corrcoef
Categorical target-rate proxies	[IMPLEMENTED]	Max-min target rate gap per category value
Future timestamp leakage	[IMPLEMENTED]	temporal_availability + training_future_date_scan
ID/proxy columns	[IMPLEMENTED]	High-cardinality + id-naming pattern scan
Train/test distribution shift	[IMPLEMENTED]	Normalised mean diff (numeric) + TVD (categorical)
JSON/Markdown/HTML output	[IMPLEMENTED]	to_json(), to_markdown(), to_html() in LeakageReport
PASS/WARN/FAIL/INSUFFICIENT_INPUT	[IMPLEMENTED]	_overall_status() logic in core.py
Explicit truth boundary	[IMPLEMENTED]	Mandatory on all output formats, cannot be suppressed
Weighted risk score	[IMPLEMENTED]	weighted_leakage_risk_score() in core.py

Section 11 -- Study Plan + Do Not Say This

11.1 30-Minute Pre-Interview Review

- Read core.py from top to bottom -- know every check's name and logic
- Be able to draw the 3-tier architecture (Name/Structure, Statistical, Temporal) from memory
- Know the 2 FAIL checks by heart: temporal_availability and training_future_date_scan
- Be able to explain TVD and why it's used instead of KL divergence
- Be able to explain the categorical proxy scan with the 'written_off' example
- Know that the weighted risk score is FAIL/high=4.5, WARN/medium=1.0
- Know INSUFFICIENT_INPUT is not PASS -- it means the check could not run
- Be ready to explain the RiskFrame and DevPulse connections
- Be ready to state the 3 leakage patterns the tool misses (nonlinear, interaction, text)

11.2 Do Not Say These Things

> These phrases will undermine your credibility. Know what not to say.

DO NOT SAY: "'FeatureLeakageLens proves there's no leakage'"

SAY INSTEAD: It flags suspicious patterns. PASS means no suspicious patterns detected by these 7 checks. Never 'no leakage exists.' The truth boundary is mandatory for this reason.

DO NOT SAY: "'It automatically removes leaky features'"

SAY INSTEAD: It never modifies the DataFrame. It produces a report. Feature removal is always a human decision because of false positive risk.

DO NOT SAY: "'It catches all types of leakage'"

SAY INSTEAD: State what it misses: nonlinear continuous leakage, feature interaction leakage, text label contamination, rolling aggregation leakage. Precision builds credibility.

DO NOT SAY: "'Pearson correlation detects all feature-target relationships'"

SAY INSTEAD: Pearson is a linear measure only. Non-monotonic and nonlinear relationships are invisible. The FUTURE mutual information check addresses this.

DO NOT SAY: "'FAIL means the model should definitely not be trained'"

SAY INSTEAD: FAIL means a confirmed temporal violation was found. Model training should be blocked until the feature is removed or the timestamps are verified. The human still reviews the specific rows and confirms the violation.

11.3 The One-Sentence Positioning Statement

> FeatureLeakageLens runs 7 checks on a DataFrame before model training -- detecting post-outcome names, high target correlation, categorical label proxies, temporal availability violations, ID proxies, and split distribution shift -- producing WARN findings for probabilistic signals and FAIL findings only for confirmed temporal violations, with a mandatory truth boundary that prevents the report from being misused as proof of no leakage.

Appendix A -- Full Demo Walkthrough: Audit on Synthetic Loan Dataset

A.1 Setup: The Demo Dataset

The demo dataset (data/demo_leakage_dataset.csv) is a synthetic loan default prediction dataset with 5,000 rows and 15 columns, including 4 deliberately injected leakage patterns. To reproduce:

```
pip install featureleakagelens git clone
https://github.com/SidharthKriplani/featureleakagelens cd featureleakagelens pip install
-e . python scripts/generate_demo_reports.py open outputs/featureleakagelens_report.html
```

A.2 Injected Leakage Patterns in the Demo Dataset

Column	Leakage Type	How Injected	Expected Check
payment_received_flag	Post-outcome name + high corr	Set to 1 when default=0 (|r|=0.94)	name_heuristic + correlation WARN
loan_id	ID proxy	Sequential integer (cardinality=1.0)	id_proxy_scan WARN
outcome_category	Categorical proxy	'written_off' category = 100% default rate	categorical_proxy_scan WARN
payment_received_ts	Temporal violation	Timestamp set after outcome_ts for 847 rows	temporal_availability FAIL
loan_region	Split distribution shift	North over-represented in train (40% vs 10% test)	split_distribution WARN

A.3 Running the Audit

The audit is configured to check all available patterns, including timestamp mapping:

```
import pandas as pd from featureleakagelens import LeakageAuditConfig, audit_dataframe df
= pd.read_csv('data/demo_leakage_dataset.csv',
parse_dates=['application_ts','outcome_ts','payment_received_ts']) config =
LeakageAuditConfig( target_col='defaulted', split_col='split',
outcome_time_col='outcome_ts', feature_time_cols={ 'payment_received_flag':
'payment_received_ts' }, high_corr_threshold=0.85,
categorical_target_rate_gap_threshold=0.65, high_cardinality_ratio_threshold=0.90, )
report = audit_dataframe(df, config) print(report.status) # FAIL report.save('outputs/')
```

A.4 Actual Report Output

Overall status: FAIL

```
Summary: rows: 5000 columns: 15 candidate_features_checked: 10 finding_count: 8
warn_count: 7 fail_count: 1 insufficient_input_count: 0 weighted_risk_score: 12.3
```

Check	Status	Feature	Evidence
post_outcome_name_heuristic	WARN	payment_received_flag	matched: ['paid', 'payment_received']
target_correlation_scan	WARN	payment_received_flag	correlation=0.940 (threshold=0.85)
categorical_proxy_scan	WARN	outcome_category	target_rate_gap=0.980 (threshold=0.65)
temporal_availability	FAIL	payment_received_flag	future_rows=847 of 4982 checked
id_proxy_scan	WARN	loan_id	unique_ratio=1.000 (threshold=0.90)
split_distribution_scan	WARN	loan_region	TVD=0.505 (threshold=0.35)
post_outcome_name_heuristic	WARN	outcome_category	matched: ['outcome']
training_future_date_scan	WARN	payment_received_ts	auto-detected datetime, 0 future training rows

A.5 Interpreting the FAIL: temporal_availability

The FAIL on temporal_availability for payment_received_flag is the most critical finding. The evidence shows 847 rows where the feature timestamp (payment_received_ts) is later than the outcome timestamp (outcome_ts). This means: for 17% of the dataset, the 'payment received' flag was recorded after the default/no-default decision was made. This flag cannot have been available at prediction time for those rows.

Correct response: remove payment_received_flag from the feature set. Do not train any model that includes this feature. If a 'payment history' signal is genuinely needed, use only payments recorded before the application date -- which requires point-in-time feature engineering from the raw transaction table.

A.6 Interpreting the WARN: categorical_proxy_scan

The categorical proxy scan found a 0.98 target rate gap on outcome_category. To understand this finding:

```
outcome_category values and default rates: 'active' : 847 rows, default_rate = 0.02 (2%) 'in_review' : 1203 rows, default_rate = 0.24 (24%) 'closed_paid' : 2156 rows, default_rate = 0.01 (1%) 'written_off' : 794 rows, default_rate = 1.00 (100%) max_rate = 1.00, min_rate = 0.01, gap = 0.99 >> threshold (0.65) The 'written_off' value is a post-outcome status: a loan is written off because it defaulted. This is pure label leakage through a categorical feature.
```

Appendix B -- LeakageAuditConfig Full Parameter Reference

Parameter	Default	When to Change	Direction
high_corr_threshold	0.85	Medical/scientific data where legitimate |r| can be >0.85	Raise to 0.90-0.95
categorical_target_rate_gap_threshold	0.65	Imbalanced datasets where even legitimate features have high gaps	Raise to 0.80
high_cardinality_ratio_threshold	0.90	UUID-format IDs (always 1.0) or low-cardinality customer IDs	Raise for UUIDs; lower for partial IDs
distribution_shift_threshold	0.35	Random splits (no shift expected) vs temporal splits (shift expected)	Lower for strict split requirements
max_unique_ratio_for_categorical_scan	0.25	To include or exclude high-cardinality categoricals from proxy scan	Lower to exclude more; raise to include zip codes etc.
train_value / test_value	'train'/'test'	When split column uses different labels	Set to actual split label values

Appendix C -- The Truth Boundary: Full Text

This text appears in all three output formats (JSON, Markdown, HTML) and cannot be suppressed. It is the complete truth boundary statement:

> FeatureLeakageLens is an auditor. WARN and FAIL findings should be reviewed against feature availability rules before removing any column. This tool flags suspicious patterns -- it does not prove leakage without business-time availability review. Leakage not detected by these checks includes: nonlinear proxy relationships, feature interaction leakage, label-in-text contamination, rolling aggregation future-peeking, and auxiliary table join leakage. Domain expert confirmation is required for every WARN finding. Only temporal availability violations (FAIL status) are unambiguous structural impossibilities that do not require confirmation before removal.

Why This Text Is Mandatory

In real deployments, audit reports get attached to model cards and passed to compliance reviewers who may not read the full report. A model card that says 'FeatureLeakageLens: PASS' without the truth boundary gets interpreted as 'no leakage exists' -- which is incorrect. The mandatory truth boundary prevents this misinterpretation by making the scope limitation visible in every output format, regardless of how the report is shared or who reads it.

Appendix D -- Comparison with Alternative Approaches

Approach	What It Catches	What It Misses	FeatureLeakageLens Advantage
Feature importance (post-training)	High-importance features worth investigating	Cannot distinguish legitimate vs leaked features; model already trained	Runs before training; no model needed
pandas-profiling / ydata-profiling	Descriptive stats, correlation heatmap	No target-specific checks; no temporal audit; no CI integration	Domain-specific checks; CI exit codes; structured findings
Manual column review	Whatever the reviewer thinks to check	Inconsistent; no documentation; non-reproducible	Systematic; documentable; attached to model card
Feature store governance	Lineage, access control, versioning	Does not audit statistical leakage patterns in existing features	Complements feature store; audits actual feature values
Mutual information scan (FUTURE)	Nonlinear relationships feature-target	Requires embedding/estimation; slower; model-dependent	FeatureLeakageLens is deterministic and zero-extra-dependency